TECHNICAL ARCHITECTURE DOSSIER

The PARJ Africa Publishing System

The end-to-end architecture behind publication at scale — author intake, editorial production, the Open Journal Systems engine, DOI registration, archival and global discovery.

5,500+	34.9k	30.3k	63.4k
JOURNAL TITLES	SUBMISSIONS	PUBLISHED ARTICLES	PUBLICATIONS

00

OVERVIEW

Contents & Executive Summary

PARJ Africa is a multi-tenant scholarly publishing platform that takes a manuscript from submission to a globally discoverable, citable, archived article. This document describes how the whole system is put together: the product surfaces, the production pipeline, the data stores, the runtime, the integrations, and the operational safeguards.

5 surfaces
Public apps & portals

2 databases
OJS + application

1 worker
Bounded Python daemon

~40 cron jobs
Lifecycle automation

Contents

01 **Whole-System Blueprint**
Block diagram & flow chart

02 **System at a Glance**
The five surfaces, one platform

03 **Context & Actors**
Who and what touches the system

04 **End-to-End Lifecycle**
Submission to discovery

05 **Application Surfaces**
App, Desk, OJS, Partner

06 **The Production Desk**
The editorial engine room

07 **Job Lifecycle & State**
States, worker, retries

08 **The OJS Publishing Engine**
Customised OJS 3.5.x

09 **DOI, Crossref & Archival**
The DOI Authority

10 **Discovery & Indexing**
Getting found everywhere

11 **Data Architecture**
Stores and canonical state

12 **Runtime & Deployment**
One VPS, many services

13 **External Integrations**
The outside world

14 **Email & Payments**
Delivery and money

15 **Caching & Performance**
Five cache layers

16 **Security & Hardening**
Locked and guarded

17 **Operations & Observability**
Keeping it alive

How to read this document. Sections 01–04 give the big picture; 05–10 follow an article through the system; 11–17 cover the seams — data, runtime, integrations and operations. Coloured bands and diagrams show flow; tables give the authoritative detail. Figures are drawn from the live estate on 15 October 2026.

01

BLUEPRINT

Whole-System Blueprint

Two views of the entire platform. A **block diagram** shows every major component and how the planes stack and connect; a **flow chart** traces a manuscript from submission through to a published, registered and globally discoverable article.

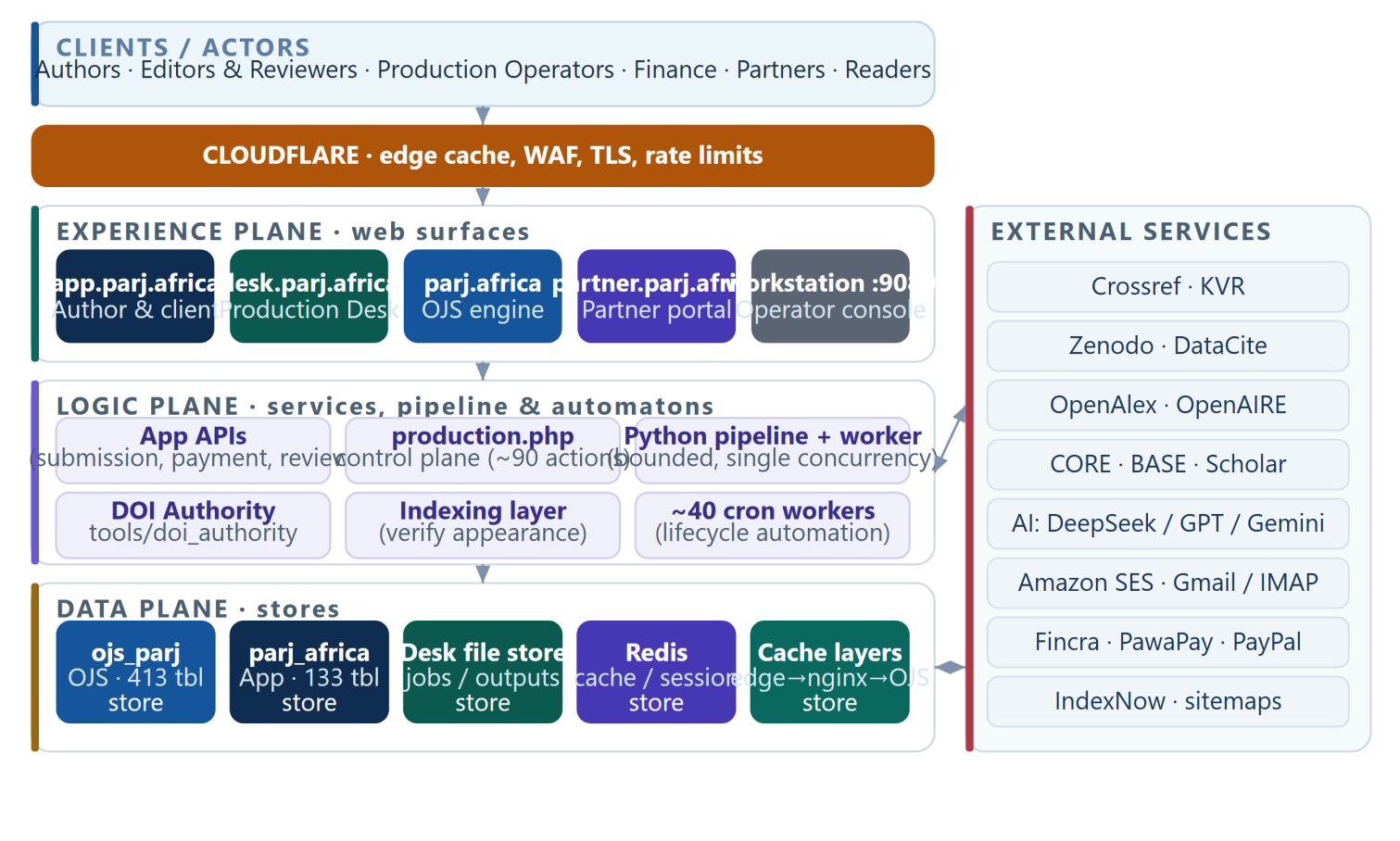


Figure 1 — Whole-system block diagram: clients, the Cloudflare edge, the three internal planes, and the external service ring.

End-to-end flow chart

The same system as a process. Every diamond is a decision point where the flow branches into an explicit, recoverable path rather than a silent failure — a failed payment is reconciled, a rejected draft is revised, an unverified hand-off is retried, and an unresolved DOI is re-synced.

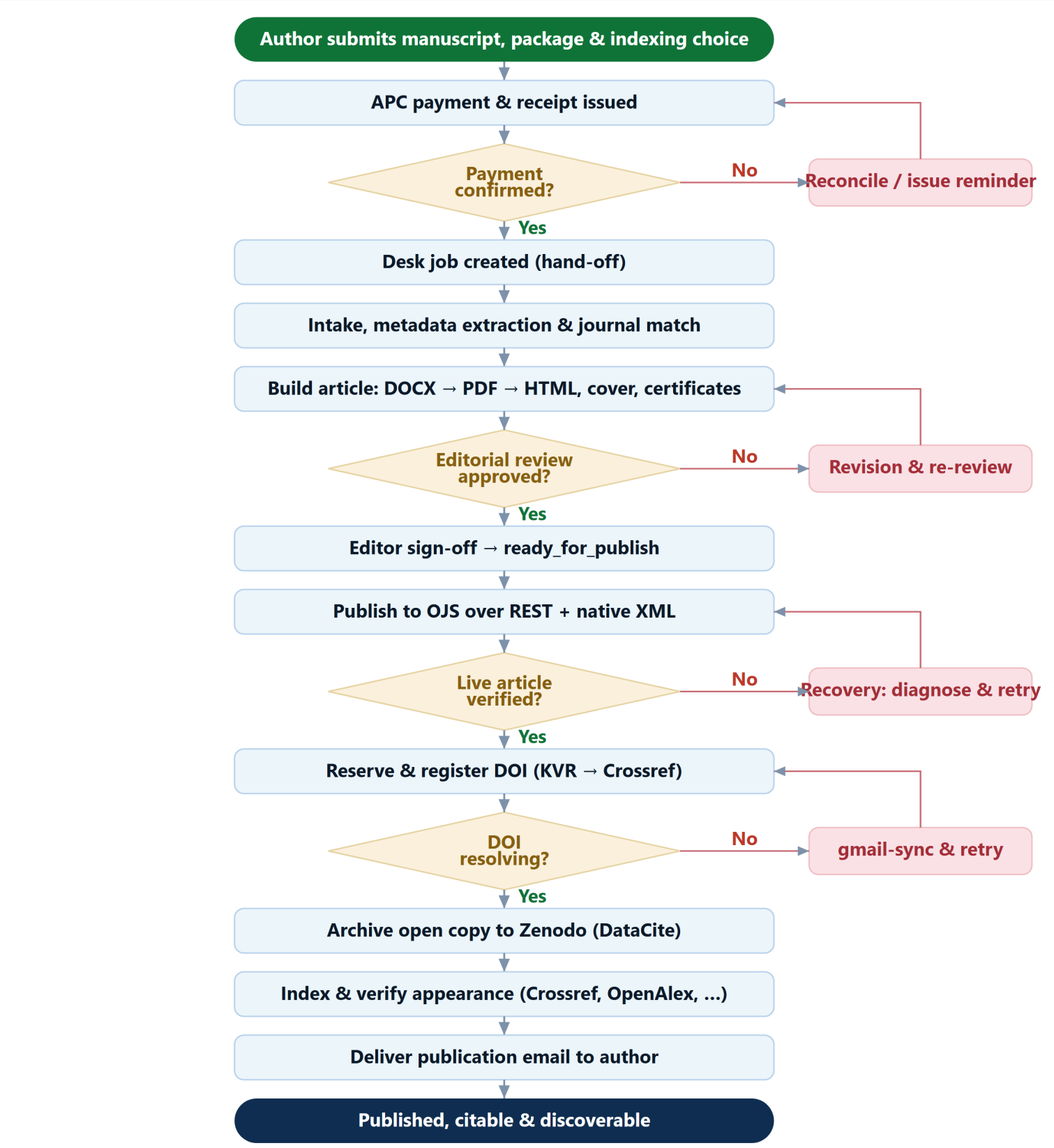


Figure 2 — Whole-system flow chart: submission to a published, citable and discoverable article.

02

BIG PICTURE

System at a Glance

PARJ is not a single application. It is a **publishing platform** composed of two large systems that meet at a well-defined boundary, wrapped by portals, automatons and integrations.

AUTHOR & CLIENT SIDE

app.parj.africa

A custom PHP portal for authors, editors, finance and support. It owns submission, peer-review coordination, the commercial layer (APCs, packages, discounts, invoices, receipts) and the growth funnel. It is the front door.

PUBLISHING SIDE

parj.africa

A heavily customised **Open Journal Systems 3.5.x** install — the academic publishing engine. It holds journals, issues, galleys and the public reading experience, and it is the system of record for published articles.

PRODUCTION

desk.parj.africa

The **Production Desk** — a PHP control plane over a Python document pipeline and a bounded worker. It turns an uploaded manuscript into a formatted article, cover, certificates and OJS/DOI/Zenodo deliverables.

PARTNERS

partner.parj.africa

A reseller and partner portal: agreements, discount codes, linked authors and manuscripts, payment requests and messaging — feeding the same core.

One platform, three planes

Experience plane

Browser apps and portals: author/client app, production desk, OJS front end, partner portal, operator workstation. HTML/CSS/JS plus PHP-rendered pages.

Logic plane

PHP service APIs (`production.php` ~90 actions; app APIs), the Python document pipeline, and ~40 scheduled workers that drive the lifecycle.

Data plane

Two MariaDB schemas, a JSON job store on disk, uploaded files and generated outputs, plus Redis and multi-layer caches.

Design philosophy. The Desk is deliberately *decoupled* from OJS. The Desk prepares and verifies a publication; it hands the finished article to OJS over two independent channels (REST and native XML). Progress is tracked in durable JSON job records, so a failed hand-off is always recoverable and never leaves a half-written article.

03

CONTEXT

Context & Actors

The system sits between people and institutions inside PARJ, and a ring of external services that provide registration, archiving, discovery, delivery and money movement.

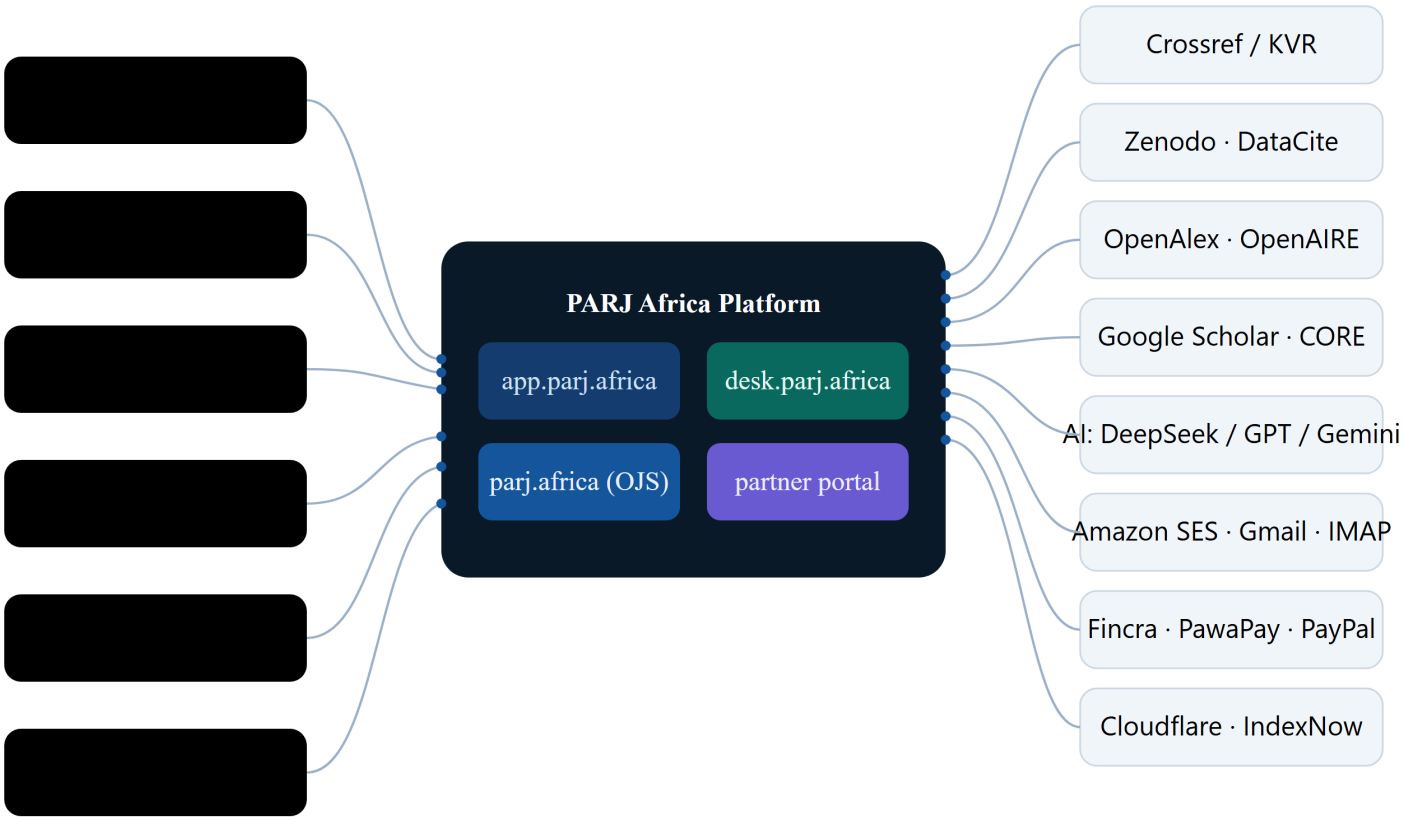


Figure 3 — PARJ core, its internal actors and the external service ring.

INTERNAL

Authors, editors, reviewers, operators, finance, partners, readers. Each is served by a purpose-built surface; all converge on the same two databases and the same article record.

REGISTRY & ARCHIVE

Crossref (via the **KVR** intermediary) registers the citable DOI; **Zenodo/DataCite** archives the open copy with its own identifier.

DELIVERY & MONEY

Amazon SES delivers mail, **Zoho/Gmail** receive it, and **Fincra, PawaPay and PayPal** move APC payments with webhook reconciliation.

04

JOURNEY

End-to-End Lifecycle

Every article travels the same road. Understanding the road is the key to understanding the architecture.

Submit → Intake → Produce → Review → Publish → Register & Discover

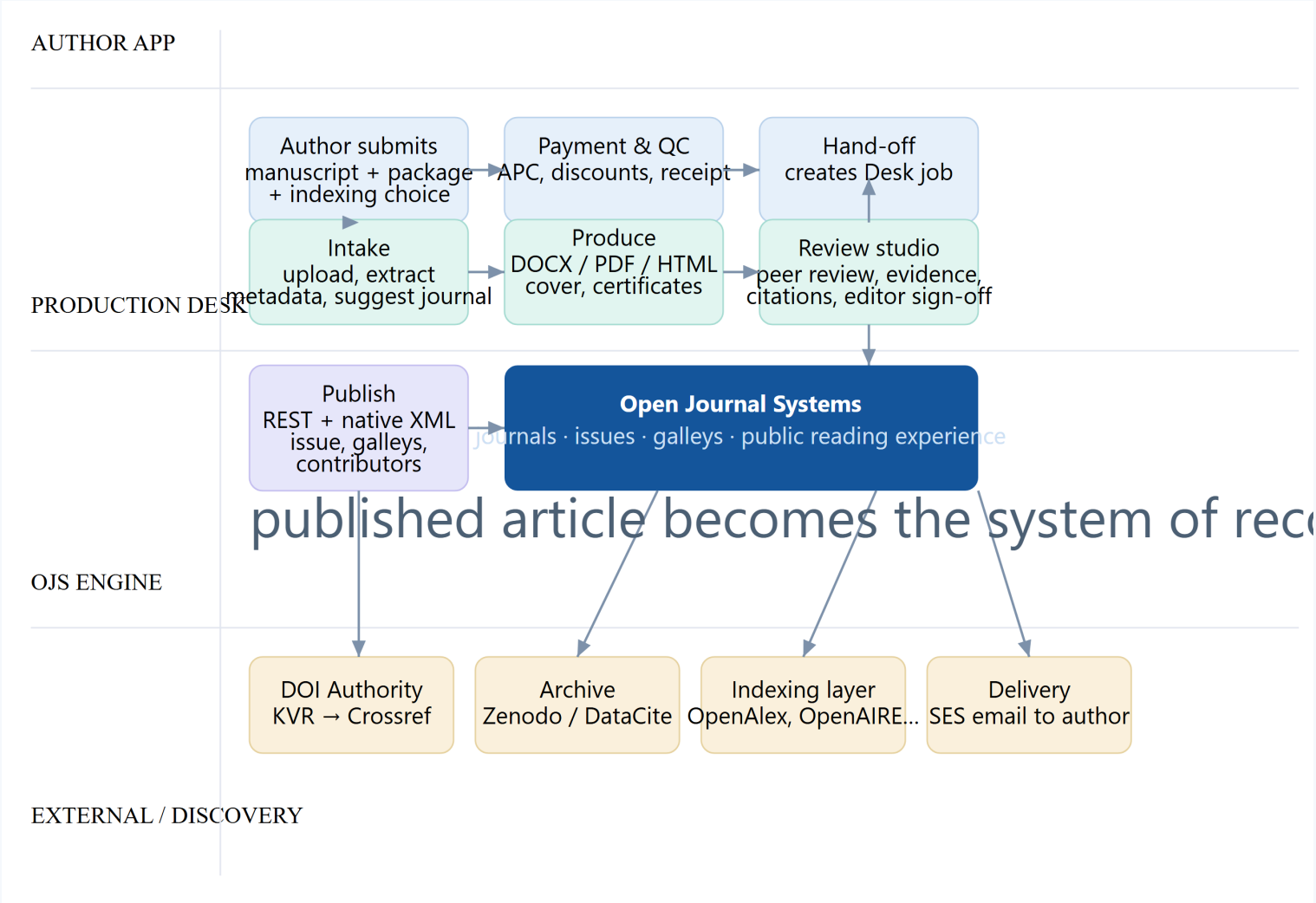


Figure 4 — The article journey across the author app, the Production Desk, OJS and the external ring.

What moves between systems

- **App → Desk:** files, metadata, package/indexing choices, tracking ID.
- **Desk → OJS:** final PDF, DOCX, HTML galley, cover, contributors, issue placement.
- **Desk → Registry:** DOI XML, resolved via KVR to Crossref.
- **Desk → Archive:** open-access package to Zenodo.
- **OJS → Discovery:** metadata, sitemaps, OAI, IndexNow pings.

The invariant

At any point, an article has exactly **one canonical state** — held as a durable JSON job record in the Desk and, once published, as a submission/publication record in OJS. Every hand-off is verified; when verification fails the article is parked in an explicit recovery state rather than being silently half-published.

05

SURFACES

Application Surfaces

Five web surfaces, each with a clear job to do. They share infrastructure but own distinct data and code.

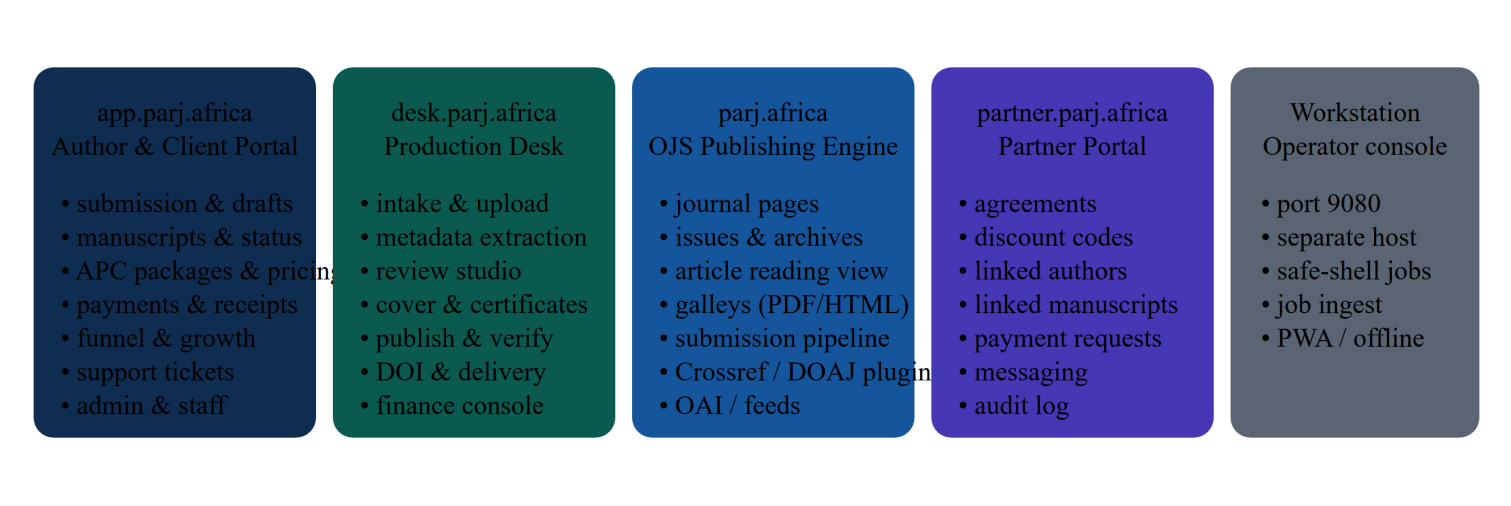


Figure 5 — The five web surfaces of the platform.

The author & client portal — app.parj.africa

A custom PHP application deployed from versioned release folders. It runs an author-facing product (submit, track, pay, receive) and the commercial and growth machinery behind it. Around 133 application tables support it.

Submission

Manuscript upload with drafts, package selection, urgency windows, indexing preferences and disciplines.

Commerce

APC packages, referral and partner discounts, invoices, receipts, failed-payment handling.

Growth

Funnel events, client follow-ups, campaigns and marketing preferences.

Support

Ticket ingestion from a mailbox, conversations, notifications and messaging.

The partner portal — partner.parj.africa

A separate PHP app and session store for resellers and institutional partners: agreements, discount codes, author/manuscript links, payment requests and a full audit trail.

06

THE ENGINE ROOM

The Production Desk

The Production Desk is where a raw manuscript becomes a finished, publishable article. It is a **PHP control plane** wrapped around a **Python document pipeline**, driven by a **bounded worker** and a large library of document and AI modules.

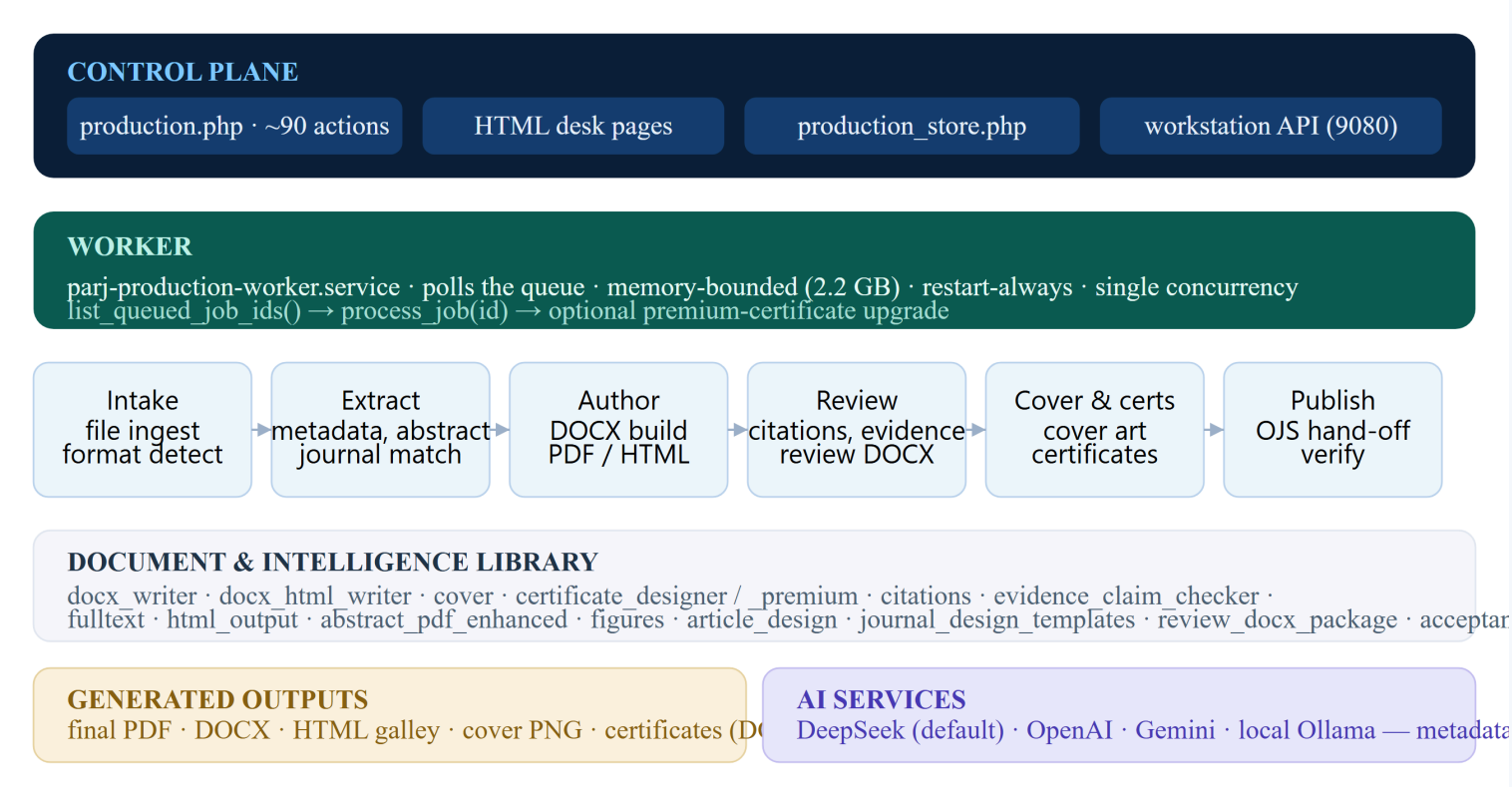


Figure 6 — The Production Desk: control plane, worker, document pipeline and output services.

What the Desk actually does

Builds the article, not just stores it

It re-typesets the manuscript into PARJ's house DOCX style, converts to PDF, produces an HTML galley, generates cover art and renders authorship/editorial certificates with QR codes.

Reviews and verifies

A review studio coordinates reviewers, produces annotated *reviewed* and *author revision* DOCX packages with tracked changes, and runs citation and evidence checks.

Publishes and proves it

It hands the finished article to OJS over REST and native XML, updates the issue and galleys, then verifies the live article before marking the job complete.

Registers and delivers

It reserves the DOI, drives registration through the DOI Authority, archives to Zenodo, and emails the author their publication package.

Why a custom Desk? OJS is optimised for editorial workflow and reading, not for high-volume re-typesetting, cover design, certificate generation and multi-channel delivery. The Desk fills exactly that gap and keeps OJS clean as the

system of record.

07 STATE

Job Lifecycle & State

Every unit of production is a **job** with a stable ID. Its state lives in a durable JSON file on disk and is mirrored into MySQL — so the Desk survives restarts and every failure is recoverable.

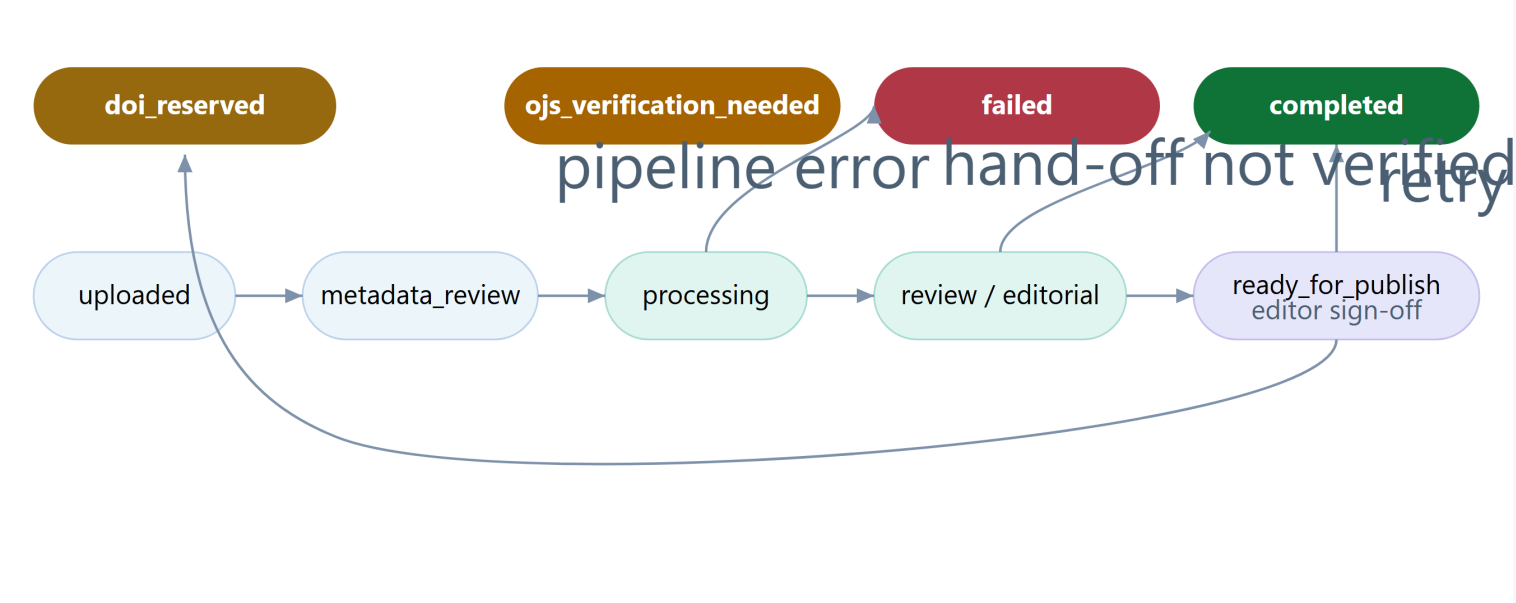


Figure 7 — Representative job states. Failures are explicit and recoverable; nothing is silently dropped.

Durable by design

- **Canonical state:** `production_data/jobs/<id>.json` — one file per job.
- **Mirror:** the `production_jobs` table for queries and dashboards.
- **Outputs:** `production_data/outputs/<id>_<slug>/` holds every generated artefact.
- **Locks:** a `locks/` area prevents two operators editing one job.

Recovery & retries

- Failed publishes keep the job and its error text; an operator retries after fixing the cause.
- An **operator recovery** concept pauses risky actions (e.g. author email) until the blocking condition clears.
- A queue watchdog sweeps for stalled jobs and resumes them.
- The worker is bounded so one bad conversion can never take down the host.

Guard-rails. The certificate renderer is an *optional* enhancement loaded lazily and gated by an environment flag — a cosmetic failure can never stall the publication queue. This is a deliberate pattern throughout the Desk.

08

SYSTEM OF RECORD

The OJS Publishing Engine

parj.africa is a **customised Open Journal Systems 3.5.x** install. It is the academic system of record: journals, issues, submissions, publications, galleys and the public reading experience. It is the largest data store in the estate, with over 400 tables.

5,534	34,923	30,264	413
journals	submissions	published	DB tables

What OJS owns

DATA MODEL

Journal → Section → Issue → Publication → Galley. An article is a *submission* with one or more *publications*; each publication carries galleys (PDF/HTML/JATS) and contributors.

PUBLIC EXPERIENCE

Journal landing pages, issue archives and the article reading view — rendered through PARJ's own theme family rather than a stock theme.

STANDARDS

OAI-PMH, JATS, Dublin Core, web feeds, and the Crossref / DataCite / DOAJ / Google Scholar plugins.

Customised, and locked

OJS has been extended in ways that must never be overwritten by a stock upgrade. PARJ therefore **locks the installation** against upgrades and documents the exact set of patched files.

LOCKED / PATCHED COMPONENT	WHY IT MATTERS
`plugins/themes/parjgeometric/`	PARJ journal homepages, footers and indexer strips across 575+ templates.
`templates/frontend/…/article_redesign.tpl`	The redesigned article page used for every published article.
`tools/ajfsDoiExport.php`	Single-publication Crossref export consumed by the DOI Authority.
`config.inc.php`	PARJ database, SMTP/SES and DOI settings.
`crossref filter (Article/Issue)`	Null-issue guards so an article published without an issue can still register.

Upgrade barrier. `show_upgrade_warning` is off, `tools/upgrade.php` is `chmod 000`, and nginx returns **403** for `/tools/` and `/index/install`. CLI use of the DOI exporter still works; only the HTTP path is blocked.

Any edit to an OJS `.php` file requires a PHP-FPM restart — opcache will otherwise keep serving the old code, and the cache layers must be purged.

09

CITABILITY

DOI, Crossref & Archival

A DOI is the article's permanent identity. PARJ mints its own DOIs under the prefix 10.68258/ and registers them with **Crossref** through the **KVR** intermediary. Independently, an open copy is archived on **Zenodo** with a DataCite identifier (10.5281/).

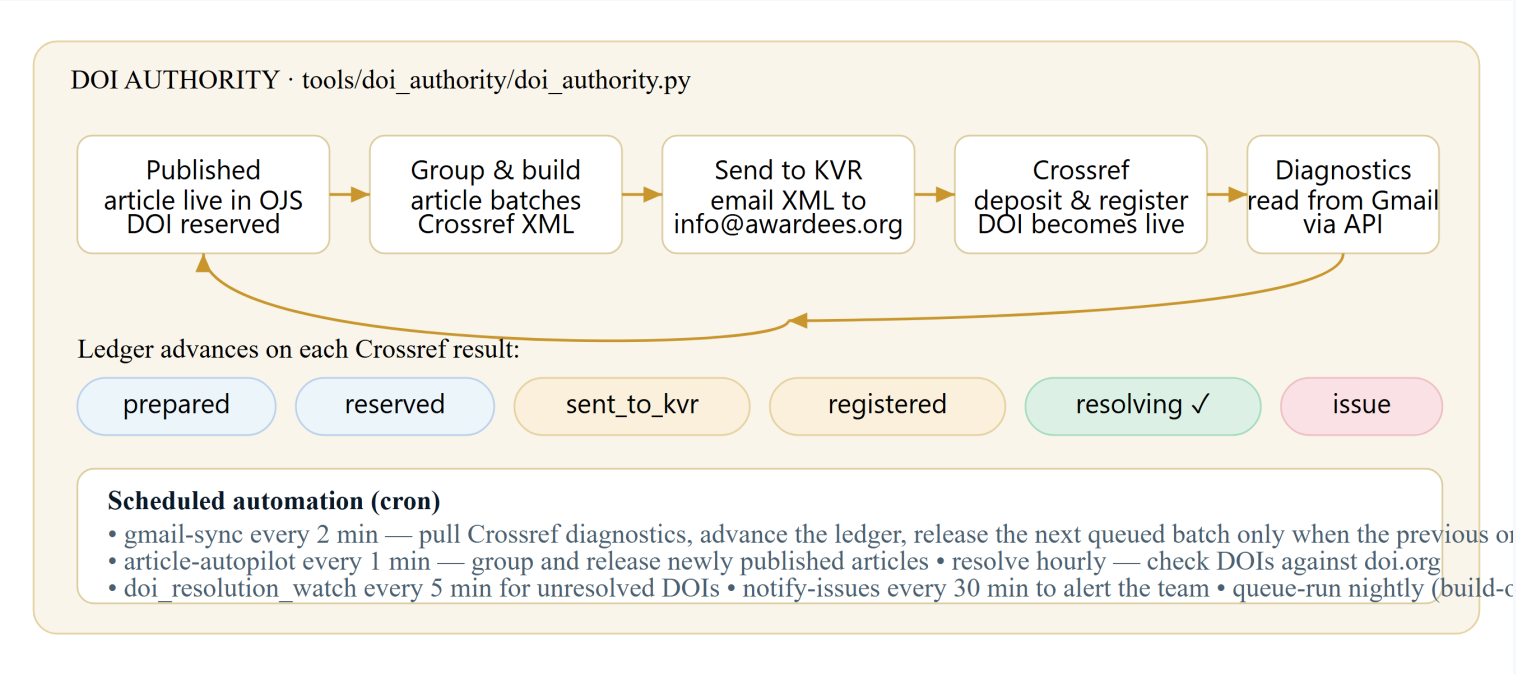


Figure 8 — The DOI Authority: a closed loop from publication to a resolving DOI.

The ledger

A DOI's lifecycle is tracked in `doi_authority_article` with supporting `doi_authority_search` (a search index) and an email log. Every DOI has an explicit state, so "reserved but not registered" is a first-class, visible condition.

Two identifiers, two jobs

Crossref DOI = the citable identity. **Zenodo DOI** = the open-access archive. PARJ policy forbids Zenodo from minting the article DOI; it archives only. The two are linked by relation metadata.

10

REACH

Discovery & Indexing

A published article is only valuable if the world can find it. A dedicated **indexing layer** at /var/www/parj.africa/indexing registers metadata, pings search engines, and — critically — **verifies** that the article has actually appeared in each external service.

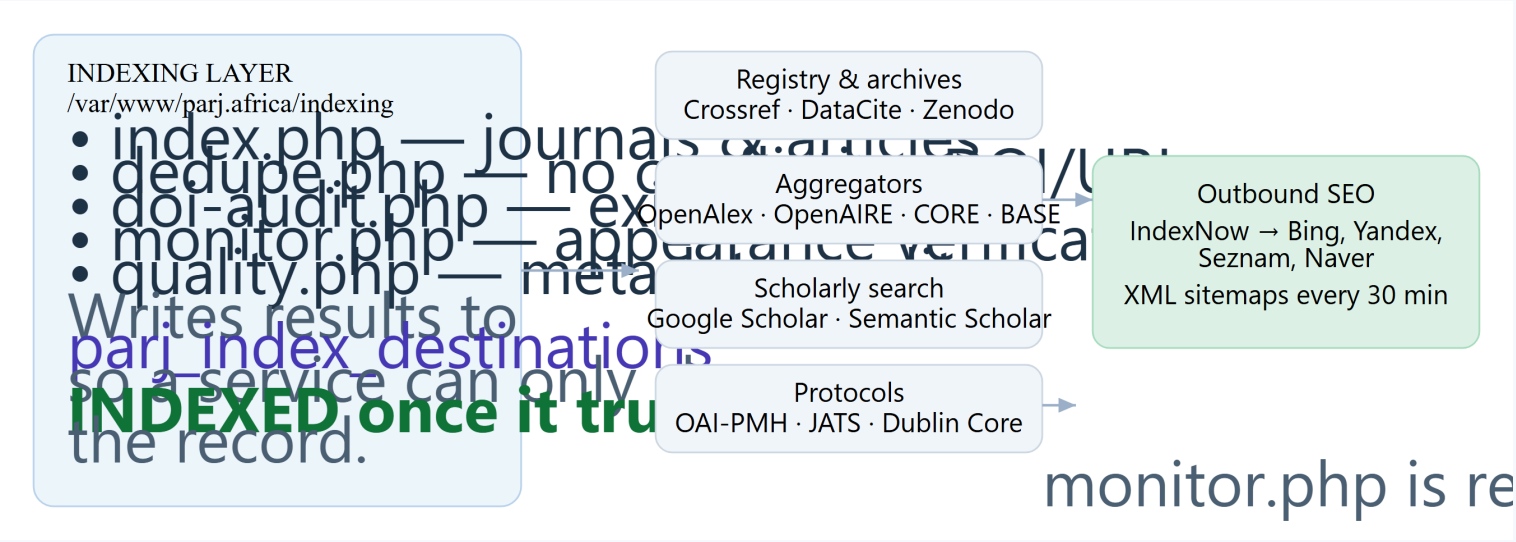


Figure 9 — Registration, aggregation, scholarly search and outbound SEO.

Honest status

The dashboard can only claim "indexed" when an external API genuinely returned the record — no optimistic self-reporting.

Frequent, cheap pings

IndexNow runs every 30 minutes and is a no-op when nothing is new, so new articles are surfaced fast and cheaply.

Health monitoring

Crossref, DataCite and OpenAlex are sampled every 6 hours; the slow OpenAIRE checks run in a small daily batch.

11

DATA

Data Architecture

The system deliberately separates **publishing state** from **commercial and production state**, and keeps the highest-churn production state on disk instead of in the database.



Figure 10 — Three stores: publishing state, application/commercial state, and production state.

Why the split

The database is the record; disk is the workbench

Publishing and commercial facts belong in the database where they can be queried and audited. But a job that rewrites a 60-page manuscript and generates a dozen artefacts dozens of times a day is far better served by a filesystem working area with a single JSON state file than by hundreds of small table writes.

Mirrored, not duplicated carelessly

The Desk mirrors each job into `production_jobs` for dashboards and cross-system queries, and hands off to the app and OJS through explicit, auditable records (`production_desk_handoffs`, OJS submissions).

Key tables by domain

A quick reference to where the important data lives and which system owns it. Publishing facts live in ojs_parj; commercial and production facts live in parj_africa.

DOMAIN	REPRESENTATIVE TABLES & STORES
Publishing (OJS)	`journals, issues, submissions, publications, publication_galleys, submission_files, files`
Submission (app)	`manuscripts, manuscript_files, manuscript_final_versions, authors, submission_drafts`
Commerce	`payments, payment_groups, payment_webhooks, pawapay_callback_events, billing_lifecycle_events`
Production bridge	`production_jobs, production_assignments, production_desk_handoffs, plus the jobs JSON files`
Peer review	`peer_reviews, review_batches, review_batch_items, ojs_review_export_queue, ojs_reviewer_pool`
DOI	`doi_authority_article, doi_authority_search, doi_authority_email_log`
Comms	`email_queue, notifications, messages, support_mail_ingest, conversations`
Growth	`funnel_events, funnel_followups, client_followup_queue, outreach_recipients, referrals`
Indexing	`parj_index_* (app DB): records appearance in each external destination`

Files, not blobs

Manuscripts, generated PDFs, DOCX files, covers and certificates are stored on the filesystem with only paths and metadata in the database. The database stays lean and fast; large binaries never bloat it.

Audit everywhere

Hand-offs, repairs, DOI events, partner actions and billing lifecycle events each write to their own audit records, so it is always possible to reconstruct what happened and when.

Backups. Database dumps and the Desk store are backed up on a schedule by the operations layer; bulky generated outputs and caches are excluded so restore stays fast and reliable.

12

RUNTIME

Runtime & Deployment

The entire estate runs on a **single Ubuntu 24.04 VPS** behind Cloudflare. Understanding this is understanding the resilience story: services are isolated, memory is bounded, and failures are contained.

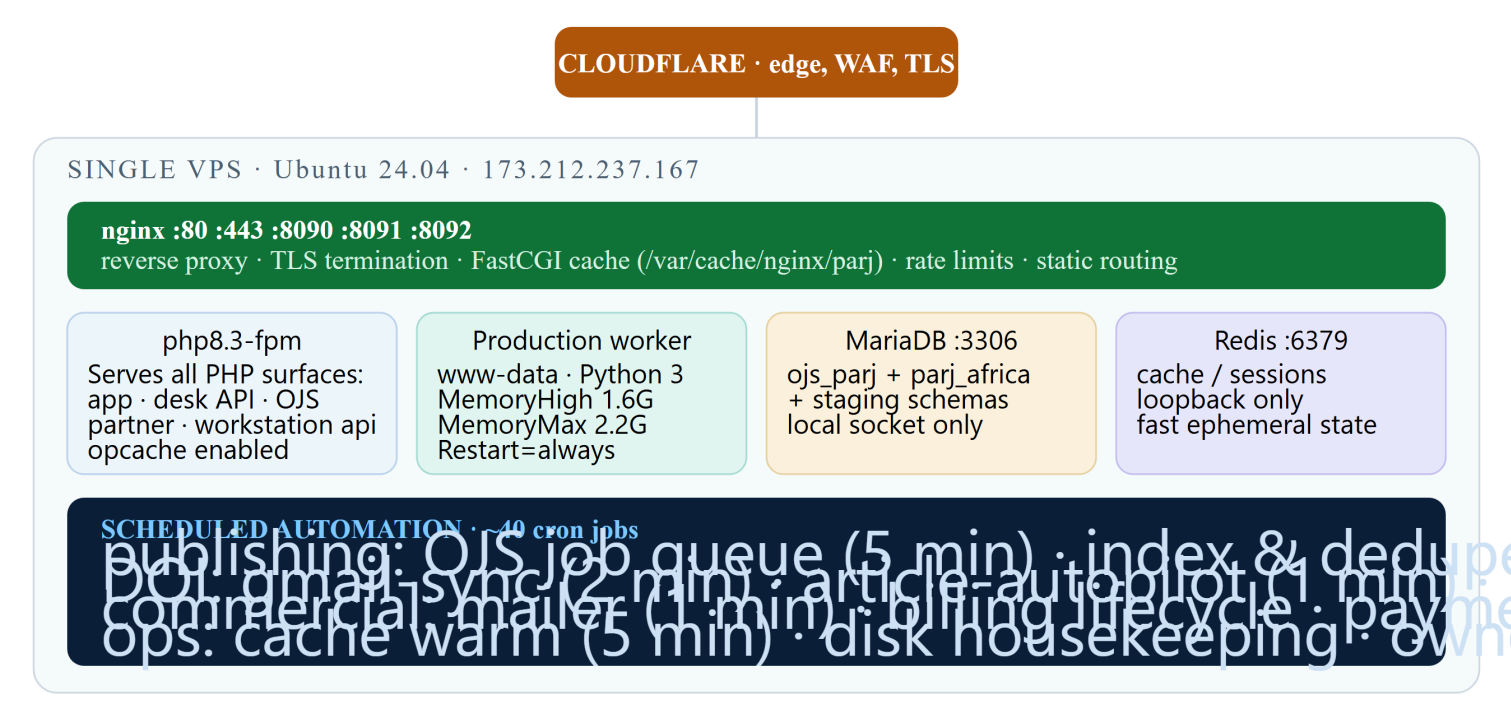


Figure 11 — Runtime topology: one host, Cloudflare at the edge, isolated service planes, and a fleet of scheduled workers.

Mail services

Postfix on loopback for relay, **OpenDKIM** signing, and **Amazon SES** as the authenticated outbound path.

Document toolchain

LibreOffice headless for DOCX→PDF, **cairosvg** for vector→raster, plus Python imaging for covers and certificates.

Background queue

OJS's own job queue is drained by a bounded cron pass every 5 minutes — no long-running daemon required.

13

OUTSIDE WORLD

External Integrations

PARJ deliberately leans on best-in-class external services rather than rebuilding them. Each integration is a defined contract with a clear owner in the system.

SERVICE	ROLE	HOW IT IS USED
Crossref 10.68258	DOI registration	Deposited via KVR; diagnostics read back from a Gmail mailbox through the Gmail API.
KVR	Registration intermediary	Receives the article XML by email and forwards to Crossref; PARJ's integration partner.
Zenodo / DataCite 10.5281	Open archive	Every published article archived under a Creative Commons licence, linked to the OJS record.
Amazon SES	Outbound email	Authenticated SMTP for all transactional and delivery mail, with a local relay fallback.
Zoho / Gmail	Inbound email	Support mailbox ingestion (IMAP) into tickets; Gmail API for Crossref diagnostics.
DeepSeek / OpenAI / Gemini	AI inference	Metadata extraction, figure and content assistance; a local Ollama fallback is available.
OpenAlex, OpenAIRE, CORE, BASE	Discovery	Metadata harvesting; appearance is independently verified by the indexing monitor.
Google Scholar, Semantic Scholar	Academic search	Scholar meta tags and web presence; tracked via the discovery monitor.
DOAJ, PubMed, Web of Science	Bibliographic exports	OJS import/export plugins for journal and article metadata.
IndexNow	Search-engine ping	Notifies Bing, Yandex, Seznam and Naver the moment an article is published.
Fincra, PawaPay, PayPal	Payments	Card and mobile-money APC payments with webhook/IPN callbacks and reconciliation.
Cloudflare	Edge & security	TLS, caching, WAF, bot mitigation and rate limiting in front of the origin.

Integration principle. Every external call is either *verified* (the system checks that the external service actually did what was asked) or *idempotent* (safe to retry). This is why a DOI is never considered live until Crossref confirms it, and an article is never considered indexed until the remote API returns it.

14

DELIVERY & MONEY

Email & Payments

Email — one queue, many purposes

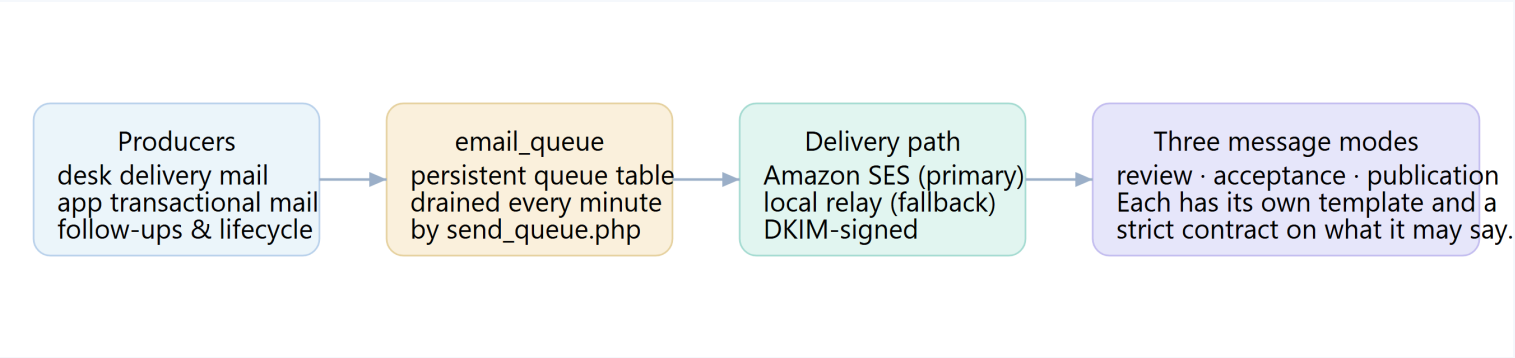


Figure 12 — A single persistent mail queue fans out to SES with a local fallback.

Mode discipline. The publication mode is the *only* template allowed to mention that an article is live, its DOI, volume, issue, archive or certificates. Review and acceptance modes must never over-promise — a rule enforced at the template layer.

Payments — gateway-agnostic, reconciled

PROVIDERS

Fincra (cards/bank), **PawaPay** (mobile money) and **PayPal**, each configurable and switchable via environment settings.

FLOW

APC → invoice → hosted payment → webhook/IPN callback recorded → payment confirmed → receipt → hand-off to production.

RECONCILIATION

Callbacks are stored, a scheduled job reconciles gateway payments every 10 minutes, and a billing lifecycle runner manages reminders and state.

15

SPEED

Caching & Performance

Because the same server renders tens of thousands of public article pages and runs heavy document jobs, performance is achieved through **layered caching**. Understanding the layers is essential, because a stale page can hide behind any one of them.

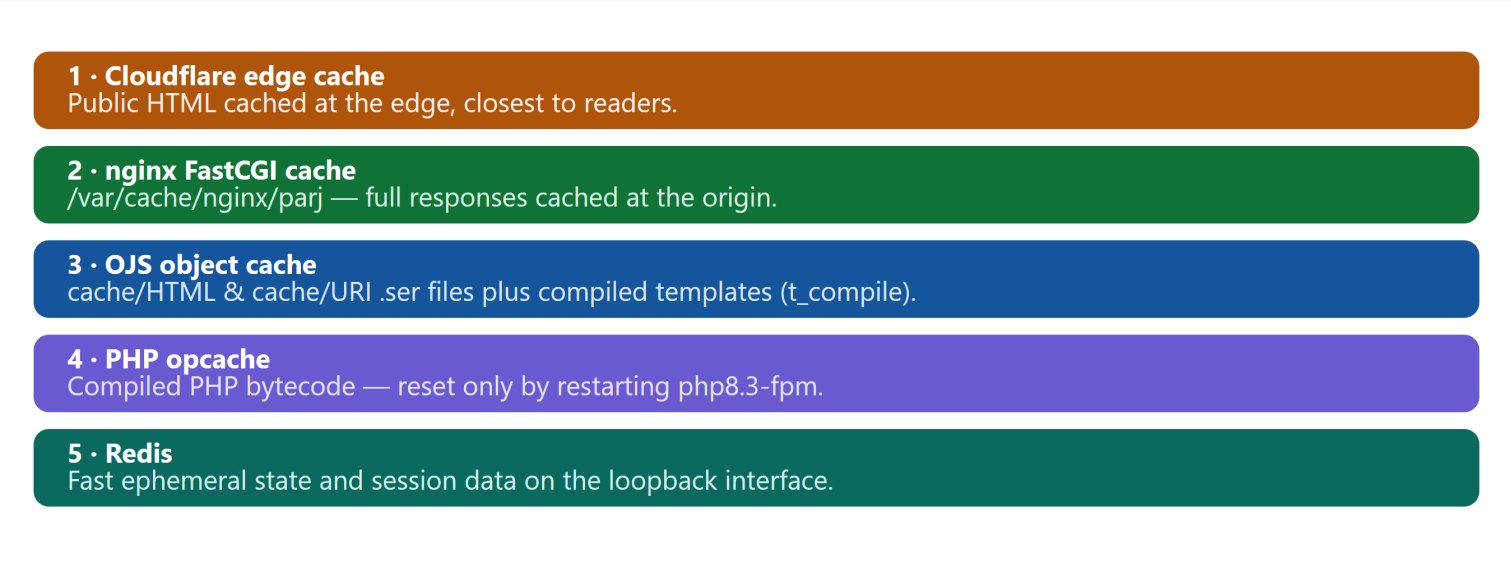


Figure 13 — Five cache layers stand between an edit and a reader.

The web-cache generator

A scheduled task pre-renders public pages into `wc-*.html` files, and an older-than-60-minute sweep deletes stale ones. It runs from a safe working directory so cleanup never fails silently.

Purge protocol

After any content change, all applicable layers are cleared together: edge, nginx cache, OJS `.ser` caches, compiled templates and the web-cache files — then PHP-FPM restarted if code changed.

16

SAFETY

Security & Hardening

The platform carries thousands of journals, tens of thousands of articles and real payments, so it is defended in depth — at the edge, at the web server, at the application and on the filesystem.

EDGE

Cloudflare

TLS, WAF and bot mitigation, with targeted rate limits for known crawlers and sensitive endpoints (search, recommend, verification, audit).

WEB SERVER

nginx rules

Denies dot-files, backup and temp files, and raw `.py` source; returns 403 for OJS `/tools/` and installer paths.

APPLICATION

Auth & roles

Separate session stores per surface, role-based access (author, editor, reviewer, production, publisher, finance, super-admin, partner).

FILESYSTEM

Guards & ownership

A scheduled ownership fixer and a file guard keep permissions correct; generated data and source are separated so web-served paths never expose internals.

INTEGRITY

Deployment lock

OJS cannot self-upgrade: the upgrade tool is disabled and the patched files are documented, protecting PARJ's customisations from being overwritten.

EMAIL

Deliverability

DKIM signing and SES authentication keep transactional mail trustworthy and out of spam folders.

Data hygiene

- **Session pruning.** Anonymous crawler sessions are purged aggressively; real users always carry an identity, so pruning can never log a person out.
- **Disk housekeeping.** Generated caches, stale sessions and temporary files are removed on schedule to prevent a full disk from taking the platform down.
- **Auditability.** Production hand-offs, repairs, partner actions and DOI events are recorded in dedicated audit tables.

Single-VPS risk, contained. Everything runs on one host. Resilience is bought by *isolation and boundedness*: the worker is memory-capped and OOM-weighted, PHP-FPM and MariaDB each have their own config, and the worker's crash cannot starve nginx or the database.

17

KEEP IT ALIVE

Operations & Observability

Running a publishing platform means running a quiet factory. The operational layer is what keeps it quiet: backups, health checks, watchdogs, reports and alerts.

Automated safeguards

- **Backups** of the Desk and databases on a schedule.
- **Hourly health checks** with email alerts on failure.
- **Queue watchdog** that detects and resumes stalled jobs.
- **Journal sync audit** against live OJS.
- **Indexing & sitemap audit.**
- **Monthly operational report.**
- **Swap provisioning** helper.

Monitoring & alerting

- A system monitor samples the host every five minutes and emails on threshold breaches.
- DOI problems are swept every 30 minutes to the admin inbox.
- Payment reconciliation flags emerge within ten minutes.
- Indexing health is sampled continuously and reported.

The operational lifecycle of a single article



The through-line. Every step above is observable, recoverable and independently verifiable. The system never says "done" unless it has proved it — a published article is confirmed live, a DOI is confirmed resolving, and an indexed article is confirmed returned by the external service.

Architecture documented from the live estate on 15 October 2026. Figures reflect the production databases and services at that time.

PARJ AFRICA PUBLISHING SYSTEM

A single, integrated platform spanning author submission, editorial production, the Open Journal Systems engine, DOI registration, archival and global discovery.

KEY NUMBERS

5 surfaces · 2 databases · 1 bounded worker · ~40 scheduled jobs · 5 cache layers · 12+ external integrations.
